CO3404 Distributed Systems

Assignment: Distributed Joke Service Architecture

Option 4 – Authentication, ECST and Continuous Deployment

Abeykoonge Vishani Nimeka

Student ID: **3000681**

UCL Sri Lanka | BSc Software Engineering

Module: CO3404 Distributed Systems

March 12, 2026

Submitted in partial fulfilment of the requirements for CO3404 Distributed Systems

Contents

1 Introduction **3**

2 System Architecture **3**

2.1 Component Overview 3

2.2 API Gateway 4

2.3 Deployment Topology 4

3 Implementation Overview **5**

3.1 Joke Service 5

3.2 Submit Service 5

3.3 Moderate Service 5

3.4 ETL Service 5

4 Event-Driven Architecture **6**

4.1 Two-Stage Moderation Pipeline 6

4.2 RabbitMQ Queues and Exchange 6

4.3 Event-Carried State Transfer 7

5 Data Management **7**

5.1 MySQL Schema 7

5.2 MongoDB Alternative 7

5.3 Runtime Database Switching 8

6 Security Implementation **8**

6.1 Auth0 OpenID Connect 8

6.2 HTTPS via Kong 8

7 DevOps and Deployment **9**

7.1 Docker Containerisation 9

7.2 Terraform Infrastructure Automation 9

7.3 Continuous Deployment Pipeline 9

8 System Resilience **10**

8.1 Individual Service Failures 10

8.2 Startup Ordering and Retry Logic 10

8.3 RabbitMQ Reconnection and State Safety 10

9 Critical Evaluation **11**

9.1 Microservices Architecture 11

9.2 RabbitMQ and Message Queuing 11

9.3 API Gateway Design 11

9.4 Event-Carried State Transfer 12

10 Self Assessment **12**

11 Conclusion **12**

References **14**

Introduction

A distributed system is a collection of independent computing nodes that cooperate over a network to provide a unified service to users. The key benefits of this approach are fault isolation, horizontal scalability, and the ability to deploy components independently without bringing down the entire platform. These properties make distributed architectures the default choice for modern web applications where continuous availability and rapid iteration are expected.

This report describes a distributed joke service built as part of the CO3404 assignment. The system allows users to browse jokes by category, submit new jokes for review, and approve or reject submissions through an authenticated moderation interface before they reach the database. Rather than building a single application server, the project distributes these concerns across four independent microservices connected through a message broker and exposed through a single API gateway.

The motivation for this design is practical: the submission and moderation workflow involves an asynchronous human step. Coupling this directly to the request-response cycle would either force the client to wait indefinitely or require polling. A message queue decouples the submit endpoint from the downstream processing, enabling the system to return an immediate acknowledgement while the moderation pipeline continues independently.

System Architecture

Component Overview

The production deployment spans four Azure virtual machines with one additional Kong VM acting as the public entry point. Table 2.1 summarises each component.

Component	Location	Responsibility
Kong API Gateway	VM4 (public IP)	TLS termination, rate limiting, routing, correlation IDs
Joke Service	VM1 – 10.0.1.4	Read API: serve jokes by type or at random
ETL Service	VM1 – 10.0.1.4	Consume approved jokes from queue; write to database
MySQL / MongoDB	VM1 – 10.0.1.4	Persistent joke and type storage
Submit Service	VM2 – 10.0.1.5	Accept submissions; publish to RabbitMQ
RabbitMQ Broker	VM2 – 10.0.1.5	Durable queues; fanout exchange for type events
Moderate Service	VM3 – 10.0.1.6	Authenticated moderator UI; approve or reject jokes

System component summary

API Gateway

Kong 3.9 runs in DB-less declarative mode, meaning the entire routing and plugin configuration is a single YAML file tracked in version control. This approach eliminates the need for a PostgreSQL database to back the gateway and makes the configuration reproducible. The YAML is rendered by Terraform before being uploaded to the VM, embedding the dynamically allocated Azure private IP addresses of the backend services.

Kong provides three key capabilities in this deployment: TLS termination using a self-signed certificate generated on the VM with an IP Subject Alternative Name, rate limiting at five requests per minute per client IP on the Joke Service read endpoint, and a global correlation-ID plugin that appends a `Kong-Request-ID` UUID header to every response. The correlation ID is directly observable in browser developer tools and provides a simple way to verify that traffic is flowing through the gateway.

Deployment Topology

Azure Standard_B1s virtual machines were selected for cost efficiency. Network Security Groups restrict SSH access to the operator's IP address while ports 80 and 443 on the Kong VM are publicly reachable. Services communicate over the private Azure virtual network without TLS between containers, which is acceptable because the private

network is not accessible from the internet. Only Kong is publicly exposed.

Implementation Overview

All four microservices are written in Node.js 18 with Express, containerised using `node:18-alpine` base images (approximately 50 MB each), and orchestrated per VM using Docker Compose. Each service owns its own database or messaging module and communicates with other services exclusively through RabbitMQ on the write path.

Joke Service

The Joke Service is a read-only API exposing three routes: `GET /joke/:type` returns a joke of a specific category, `GET /joke/any` returns a random joke, and `GET /types` returns the current list of categories. The service selects a random joke using `ORDER BY RAND() LIMIT 1` in MySQL or the `$sample` aggregation stage in MongoDB, depending on the active database adapter.

Submit Service

The Submit Service exposes `POST /submit` and `GET /types`. On a valid submission, the joke is serialised to JSON and published to the `submit` RabbitMQ queue with `durable: true` and `persistent: true` flags, ensuring the message survives a broker restart. The service returns HTTP 202 immediately, decoupling the client from the downstream pipeline. The `/types` endpoint reads from a locally cached JSON file maintained by the ECST mechanism, so it remains functional even when VM1 is unreachable.

Moderate Service

The Moderate Service is an Express application serving a browser-based moderation UI. It consumes from the `submit` queue using the AMQP basic-get method with a channel prefetch of one, ensuring only one joke is in-flight at any time. An authenticated moderator can approve a joke, which publishes it to the `moderated` queue with the reviewer's Auth0 email attached as `moderatedBy`, or reject it, which permanently discards the message with `nack(false, false)`. Authentication is handled by Auth0 OpenID Connect, described in Section 6.

ETL Service

The ETL (Extract, Transform, Load) Service is a headless consumer reading from the `moderated` queue. It inserts the joke type into the `types` table (or MongoDB

collection) using an idempotent upsert, retrieves or creates the canonical type ID, and inserts the joke into the `jokes` table. A message is acknowledged only after a confirmed database write. Malformed JSON is nacked with requeue disabled; transient database failures cause the message to be requeued for automatic retry. After each new type insertion, the ETL publishes an ECST event as described in Section 4.

Event-Driven Architecture

Two-Stage Moderation Pipeline

The submission pipeline passes through two sequential queues before a joke reaches the database:

1. The user submits a joke via the Submit Service, which publishes to the `submit` queue and returns HTTP 202 immediately.
2. A moderator fetches the joke from the `submit` queue through the Moderate Service UI.
3. Approval publishes the joke to the `moderated` queue. Rejection discards it permanently.
4. The ETL Service consumes from `moderated` and writes the joke to the database.
5. The joke becomes visible through the Joke Machine read API.

This architecture enforces a human review gate at the infrastructure level: it is architecturally impossible for an unreviewed joke to reach the database, because the ETL Service only reads from the `moderated` queue, not the `submit` queue. This is a stronger guarantee than an application-level permission check, which could be bypassed by a code defect.

RabbitMQ Queues and Exchange

Both queues (`submit` and `moderated`) are declared as durable with per-message persistence. In addition, a `type_update` fanout exchange is declared for the ECST mechanism. A fanout exchange delivers a copy of each published message to every bound queue without any routing key matching, making it the natural choice for a broadcast pattern.

Event-Carried State Transfer

Event-Carried State Transfer (ECST) is a messaging pattern in which a producer publishes its full current state as an event so that consumers can maintain a local replica rather than polling. In this system, the ETL Service publishes the complete updated joke type list to the `type_update` exchange after every new type insertion. RabbitMQ fans this out to two bound queues: `sub_type_update` (consumed by Submit) and `mod_type_update` (consumed by Moderate). Each subscriber writes the received JSON to a file on a named Docker volume, which persists across container restarts.

The practical benefit is that the `/types` dropdown in both the Submit and Moderate UIs remains accurate and available even if the Joke Service is restarted or temporarily unreachable, because there is no live HTTP dependency. The trade-off is that the cached type list reflects the last received event rather than the live database state, introducing a brief period of eventual consistency when a new type is added.

Data Management

MySQL Schema

The default database backend is MySQL 8.0. The schema comprises two tables:

- `types(id INT AUTO_INCREMENT, name VARCHAR(100) UNIQUE)` – stores joke categories with a unique constraint on the name.
- `jokes(id INT AUTO_INCREMENT, setup TEXT, punchline TEXT, type_id INT, FOREIGN KEY(type_id) REFERENCES types(id))` – stores joke content with a foreign key linking to the category.

Type resolution uses `INSERT IGNORE INTO types (name) VALUES (?)` followed by `SELECT id FROM types WHERE name = ?`. The unique constraint makes this pattern safe under concurrent inserts because a duplicate insert silently fails rather than raising an error, and the subsequent select retrieves the canonical ID regardless of which concurrent writer inserted it.

MongoDB Alternative

MongoDB 7.0 is available as an alternative backend, selected by setting the `DB_TYPE=mongo` environment variable. Jokes are stored as documents in a `jokes` collection with `setup`, `punchline`, and `type` fields. The random joke endpoint uses the `$sample` aggregation stage rather than `ORDER BY RAND()`. Type detection uses `findOneAndUpdate` with

`upsert: true` and the `$setOnInsert` operator, which is the document-store equivalent of `INSERT IGNORE`: it creates the document if absent or returns the existing one if present.

Runtime Database Switching

The two database adapters share identical exported function names, so all service code above the adapter layer is unaffected by the active backend. Docker Compose profiles (`mysql` and `mongo`) start the corresponding database container alongside the application services. This design makes it straightforward to compare relational and document-store behaviour on the same application workload without modifying any application code.

Security Implementation

Auth0 OpenID Connect

The Moderate Service protects its moderation interface using Auth0 as an external OpenID Connect (OIDC) identity provider. The `express-openid-connect` library handles the complete authentication flow: the library redirects unauthenticated users to Auth0's hosted login page, Auth0 issues an authorisation code after successful login, the library exchanges this for an ID token and access token via the Auth0 token endpoint, validates the token's signature and claims, and establishes an encrypted session cookie.

Setting `authRequired: false` globally means the health check and public informational endpoints remain accessible without authentication. Individual sensitive endpoints (`POST /moderated`, `POST /reject`, `GET /moderate`) are protected by a custom `requiresAuthJson()` middleware that detects `fetch()` requests and returns a structured `{error: "Authentication required"}` JSON body with HTTP 401, rather than issuing an HTML redirect. An HTML redirect would cause the JavaScript frontend to receive a login page as a 200 response, leading to silent failures that are difficult to debug.

The moderator's email address is extracted from the Auth0 ID token and attached to every approved joke as the `moderatedBy` field, providing an audit trail in the database.

HTTPS via Kong

Kong terminates TLS for all public traffic. The certificate is a self-signed certificate with an IP Subject Alternative Name (SAN), generated on the Kong VM during Terraform provisioning using OpenSSL 3. The IP SAN is required because modern browsers and

`curl` validate certificates against the SAN extension rather than the legacy Common Name field; a certificate without an IP SAN would be rejected even if the Common Name matches. All internal traffic between Kong and backend services travels over the Azure private network without TLS, which is appropriate because the private network is not reachable from the internet.

DevOps and Deployment

Docker Containerisation

Each microservice has a dedicated Dockerfile based on `node:18-alpine`, producing images of approximately 50 MB. The `COPY package*.json ./` and `RUN npm install` instructions appear before `COPY . .` in each Dockerfile so that the dependency installation step is a cached Docker layer that only re-runs when dependencies change – a standard optimisation that significantly reduces rebuild times during development. The `restart: unless-stopped` policy on all containers ensures automatic recovery from process crashes or host VM reboots.

Terraform Infrastructure Automation

Terraform automates the Kong VM lifecycle using the `azurerm` provider. A `null_resource` block with a `remote-exec` provisioner installs Docker on the VM. A subsequent `file` provisioner uploads the Docker Compose file and the Kong configuration, where the YAML is rendered from a Terraform template file (`templatefile()`) that substitutes the Kong VM's dynamically allocated public IP and the backend VMs' private IPs. A final `local-exec` provisioner retrieves the generated TLS certificate back to the local machine for browser trust configuration.

The backend VMs (VM1–VM3) are provisioned separately using per-VM setup scripts (`vm1-setup.sh`, `vm2-setup.sh`, `vm3-setup.sh`) and Docker Compose files, with environment variables supplied from `.env` files per VM. This separation keeps the Terraform scope limited to the Kong gateway, which benefits from being fully torn down and reprovisioned between test cycles.

Continuous Deployment Pipeline

A GitHub Actions workflow file (`.github/workflows/deploy-moderate.yml`) implements a two-stage CI/CD pipeline for the Moderate Service. The pipeline triggers only on pushes to the `main` branch that modify files under `moderate-service/**`, using the `paths` filter to avoid unnecessary builds caused by unrelated commits.

The `build-push` job uses the official `docker/build-push-action` to build and push the Moderate Service image to Docker Hub, tagged with both `latest` and the short commit SHA. The `deploy` job depends on `build-push` and SSHs into VM3 using `appleboy/ssh-action`, pulling the commit-SHA-tagged image and restarting the container with `docker compose up -force-recreate -d`. Docker Hub credentials and the VM3 SSH private key are stored as GitHub Actions Secrets, ensuring they never appear in version control.

System Resilience

Individual Service Failures

The asynchronous queue design provides natural resilience to individual component failures. If the Submit Service becomes unavailable, Kong returns an error to the submitting user, but messages already queued in RabbitMQ are durably stored and continue to accumulate. If the Joke Service restarts, the Submit and Moderate UIs continue serving the most recently received ECST type cache for the duration of the outage. If the Moderate Service is temporarily unavailable, submissions queue up in the `submit` queue; on recovery, the moderator continues from where processing stopped with no data loss. If the ETL Service crashes mid-write, the unacknowledged `moderated` message is re-delivered by RabbitMQ on reconnection, guaranteeing at-least-once delivery to the database.

Startup Ordering and Retry Logic

Containerised systems have a well-known startup race: even after a Docker healthcheck passes, the database may require additional seconds before accepting application connections. The ETL and Joke Services implement a `connectWithRetry()` function that attempts reconnection up to ten times at three-second intervals before terminating. This complements the Docker Compose `depends_on: condition: service_healthy` directive, which handles gross ordering but cannot guarantee application-level readiness.

RabbitMQ Reconnection and State Safety

All three RabbitMQ-connected services implement an automatic reconnection loop. The Moderate Service additionally resets its `pendingMsg` and `pendingJoke` state variables on reconnection to prevent a ghost-approval scenario, where a message handle held from before a connection drop is acknowledged after the connection is restored, incorrectly approving a joke that may have already been re-queued and delivered elsewhere.

Critical Evaluation

Microservices Architecture

The microservices design enables independent deployment, fault isolation, and separate scalability for each component. A moderator UI outage does not affect the Joke Machine read path. A read spike on the Joke Service can be handled by scaling that service alone.

The main cost is operational complexity. Four services, four Docker Compose files, two queue stages, a fanout exchange, an API gateway, and four virtual machines are considerably more infrastructure than a monolith would require. For a team already familiar with container operations, this overhead is manageable; for a team new to the stack, it increases the surface area for misconfiguration. For this academic project, the complexity is a deliberate learning objective.

RabbitMQ and Message Queuing

The two-stage queue pipeline enforces the moderation gate at the infrastructure level rather than in application code, which is a robust architectural decision. The AMQP acknowledgement model provides at-least-once delivery with correct application-side nack handling for both malformed and transient-error cases.

The main limitation is the absence of a dead-letter exchange: a message that fails consistently for a non-transient reason will be re-queued indefinitely rather than moved to a quarantine queue for inspection. Apache Kafka would offer stronger ordering guarantees and message replay, but its operational overhead would far outweigh the benefit for this workload. Redis Streams would be a lighter-weight alternative if the team already operated Redis.

API Gateway Design

Kong's DB-less mode is operationally clean: the full gateway state is a single version-controlled YAML file applied atomically on deployment. Rate limiting on the read endpoint and the global correlation-ID plugin are practical additions that go beyond basic proxying.

The self-signed TLS certificate is the most significant usability weakness, as users must manually accept a browser security warning. A registered domain name with a Let's Encrypt certificate would resolve this. Traefik would provide native Docker Compose integration and automatic Let's Encrypt support, but Kong's plugin ecosystem is

substantially richer for enterprise requirements.

Event-Carried State Transfer

ECST eliminates the polling dependency between three services, improving resilience. The trade-off is eventual consistency: the cached type list reflects the last event, not the live database state. The lag is sub-second in normal operation and the type list changes rarely, making this trade-off acceptable. A shared Redis cache would provide faster propagation with TTL-based expiry, at the cost of introducing an additional dependency.

Self Assessment

This submission targets the **First Class** grade band by implementing all requirements cumulatively across Options 1 through 4.

Option 1 requirements are satisfied by the functioning Joke, Submit, and ETL Services backed by MySQL, connected through durable RabbitMQ queues, and containerised with Docker Compose. **Option 2** requirements are satisfied by Kong in DB-less mode with rate limiting, correlation IDs, HTTPS, and Terraform automating a four-VM Azure deployment including dynamic TLS certificate generation. **Option 3** requirements are satisfied by the Moderate Service with Auth0 OIDC authentication and the ECST type propagation mechanism that eliminates polling dependencies. **Option 4** requirements are satisfied by MongoDB as a runtime-switchable database backend and the GitHub Actions CI/CD pipeline for the Moderate Service.

The architecture is justified throughout this report with reference to design alternatives and the distributed systems principles underlying each decision. The ECST pattern, the two-stage queue moderation gate, and the idempotent ETL acknowledgement semantics demonstrate understanding of distributed systems concepts beyond basic request-response integration. On these grounds, the submission justifies a First Class assessment.

Conclusion

This project implements a distributed joke service that demonstrates a range of distributed systems principles in a practical, cloud-deployed application. The two-stage RabbitMQ moderation pipeline enforces a human review gate without coupling the submit endpoint to downstream processing latency. Event-Carried State Transfer

eliminates synchronous polling dependencies between three services, improving both performance and resilience. Auth0 OpenID Connect delegates authentication to a managed identity provider, removing the operational risk of self-managed credentials. The runtime-switchable MySQL/MongoDB adapter demonstrates how the same application can operate over different data models without code changes. Kong API Gateway in DB-less mode provides a clean, version-controlled entry point with rate limiting, TLS, and observability. Terraform automates cloud infrastructure provisioning reproducibly, and GitHub Actions delivers code changes to production without manual steps.

The central architectural trade-off is that loose coupling through durable queues produces a more resilient system at the cost of eventual consistency and operational complexity. For a content submission pipeline where data integrity matters more than read-your-writes consistency, this is the right trade-off. The system demonstrates that distributed architecture, when applied with clear design intent, produces software that is more robust and independently deployable than a monolithic alternative.

References

[1] Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems*. 2nd edn. Sebastopol, CA: O'Reilly Media.

[2] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Shelter Island, NY: Manning Publications.

[3] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media.

[4] RabbitMQ Documentation (2024). *AMQP 0-9-1 Model Explained*. Available at: https://www.rabbitmq.com/tutorials/amqp-concepts [Accessed 12 March 2026].

[5] Kong Inc. (2024). *Kong Gateway – DB-Less and Declarative Configuration*. Available at: https://docs.konghq.com/gateway/latest/production/deployment-topologies/db-less-and-declarative-config/ [Accessed 12 March 2026].

[6] Auth0 (2024). *express-openid-connect – Node.js SDK for OpenID Connect*. Available at: https://github.com/auth0/express-openid-connect [Accessed 12 March 2026].

[7] HashiCorp (2024). *Terraform – Infrastructure as Code*. Available at: https://developer.hashicorp.com/terraform/intro [Accessed 12 March 2026].

[8] Fowler, M. (2017). *Event-Carried State Transfer*. Martin Fowler's Blog. Available at: https://martinfowler.com/articles/201701-event-driven.html [Accessed 12 March 2026].